스킨케어 추천 챗봇 고도화 이론 보고서

Nov 11, 2026 · @용

1. 개요

이번 고도화는 스킨케어 추천 챗봇이 고객의 실제 말투를 이해하고, 최신 정보를 답하고, 위험한 입력을 막도록 세 곳을 보강한 작업이다. 여기에 제품 성분 데이터를 보완해 추천의 근거를 단단하게 했다.

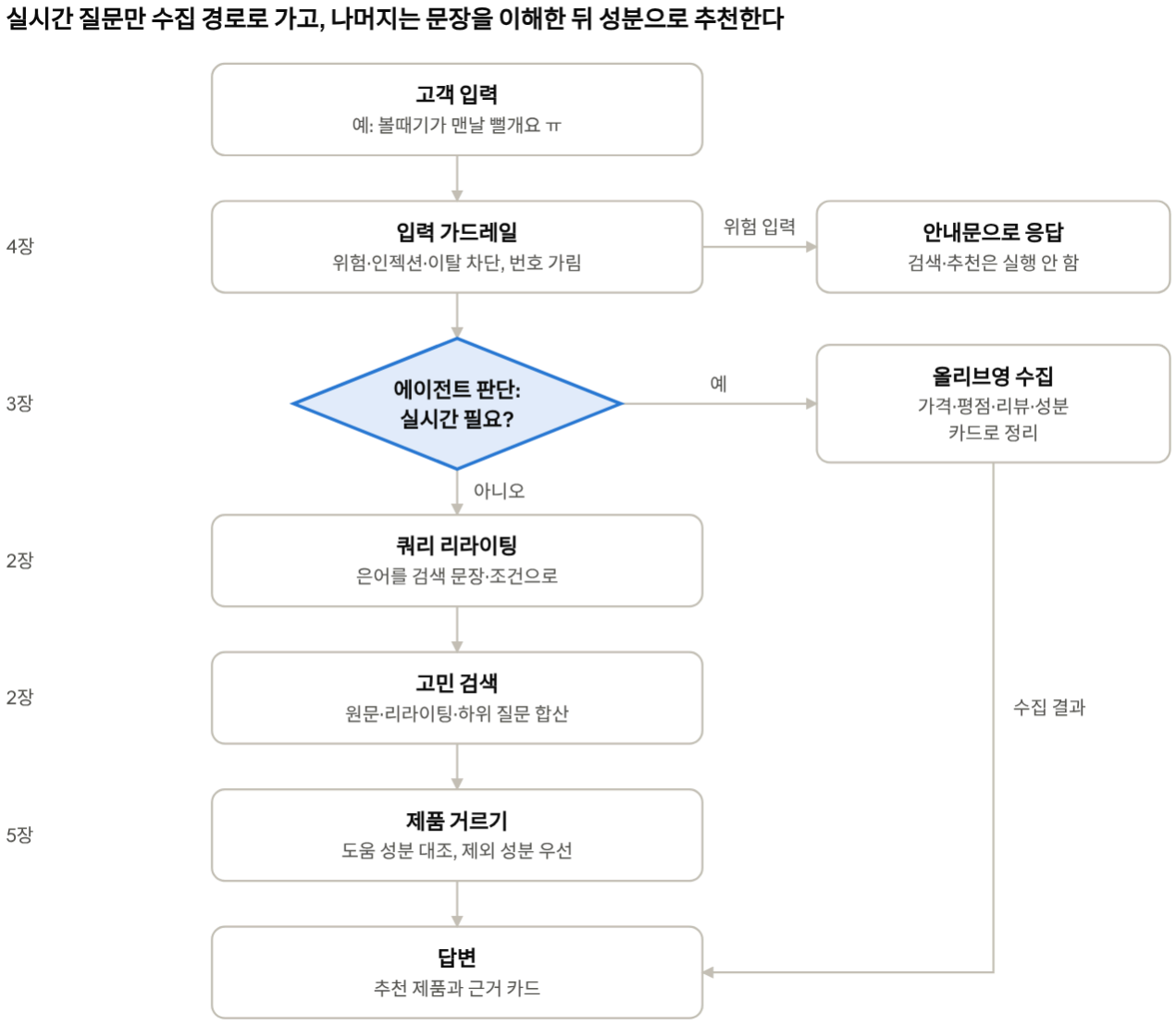
기존 시스템과 한계. 기존 챗봇은 고객 문장을 임베딩 모델(KURE-v1, BGE-M3)로 검색해 피부 고민을 찾고, 그 고민에 맞는 성분이 든 제품을 추천했다. 이 구조에는 네 가지 한계가 있었다.

- 은어·오타("볼때기가 맨날 뻘개요")는 고민 설명과 글자도 의미도 멀어 검색이 흔들린다.
- 피부 타입, 제품 종류, 피하고 싶은 성분, 임신 여부처럼 문장에 섞인 조건을 따로 뽑아 추천에 쓰는 장치가 없었다.
- 순위·가격처럼 매일 바뀌는 질문은 저장된 데이터만으로 답할 수 없다.
- 입력을 검사하는 단계가 없어, 응급 증상이나 프롬프트 인젝션 같은 입력이 그대로 모델에 닿는다.

이번 개선. 네 가지 개선은 서로 다른 한계를 맡고, 파이프라인의 서로 다른 위치에 들어간다.

개선	해결하는 문제	핵심 기법	파이프라인 위치
가드레일	위험 증상, 프롬프트 인젝션, 주제 이탈, 개인정보	규칙 기반 사전 검사·마스킹	가장 앞 (RAG 이전)
올리브영 크롤링 에이전트	순위·가격 같은 최신 정보	LangGraph 상태 그래프, 필요성 판단 후 수집	검색 앞 분기
쿼리 리라이팅	은어·구어체, 문장에 섞인 조건	LLM 도구 호출(tool_use)로 구조화된 재작성	고민 검색 직전
성분 데이터 보완	성분표 부족으로 약했던 추천 근거	고민-성분 매핑, 전성분 대조	제품 거르기

전체 처리 순서는 아래 그림과 같다. 각 단계의 이론과 설계는 2~5장에서 다룬다.



전체 처리 흐름 · 분기는 가드레일과 에이전트 두 곳

입력은 가드레일을 먼저 통과해야 한다. 통과한 뒤 실시간 정보가 필요한 질문만 수집 경로로 가고, 나머지는 리라이팅·검색·거르기를 거쳐 답변이 된다. 왼쪽 숫자는 해당 단계를 다루는 장이다.

2. 쿼리 리라이팅

쿼리 리라이팅은 고객의 문장을 검색기가 잘 알아듣는 문장으로 바꿔 검색하는 기법이고, 이 시스템에서는 Claude가 그 일을 맡는다. 검색 증강 생성(RAG)은 외부 지식을 검색해 답변 생성에 쓰는 구조인데(Lewis et al., 2020), 입력 문장과 실제로 필요한 지식 사이에는 늘 간극이 있다. Ma et al.은 이 간극을 검색 앞단에서 줄이는 Rewrite-Retrieve-Read 구조를 제안했다(Ma et al., 2023). 이번 설계는 같은 발상을 피부 고민 검색에 적용한 것이다.

2.1 왜 필요한가

기존 검색은 고객 문장과 각 고민의 설명문을 임베딩 모델로 비교한다. KURE-v1은 BGE-M3를 한국어 데이터로 추가 학습한 검색 모델이다(KURE-v1 모델 카드, Chen et al., 2024). 그래도 "볼때기가 맨날 뻘개요 ㅠ"처럼 은어·축약·이모티콘이 섞인 문장은 "볼이 늘 붉어져요" 같은 표준 표현보다 설명문과 멀어질 수 있다. 또 문장에 함께 들어 있는 조건(피부 타입, 제품 종류, 피할 성분, 임신 여부)은 임베딩 점수에는 드러나지 않는다. 리라이팅은 두 문제를 한 번의 호출로 푼다.

2.2 Claude에게 시키는 일: 구조화 추출

Anthropic API는 `tool_choice`로 특정 도구를 지정해 그 도구를 쓰도록 강제할 수 있다(Anthropic 문서). 이 시스템은 실제로 실행할 도구가 없는데도 `emit_rewrite`라는 도구를 강제 호출한다. 도구의 입력 스키마가 곧 출력 형식이 되어, 자유 문장 대신 정해진 필드가 돌아온다.

필드	뜻	쓰이는 곳
rewritten	은어·구어를 걷어 낸 검색용 한 문장	고민 검색
sub_queries	고민이 여럿일 때 고민별로 나눈 문장 (최대 3개)	고민 검색
skin_type	건성·지성·복합성 중 하나 또는 없음	제품 거르기
product_type	정해진 제품 종류 중 하나 또는 없음	제품 거르기
avoid	피하고 싶은 성분 (최대 8개)	제품 거르기
pregnant, sensitive	임신·수유 여부, 민감성 여부	안전 규칙
needs_clarification	너무 모호하면 되묻기	응답

2.3 모델의 답을 그대로 믿지 않는 장치

LLM은 실패하거나 엉뚱한 값을 낼 수 있어서, 돌아온 값을 코드가 한 번 더 검사한다.

- **스키마 제약:** 피부 타입과 제품 종류는 정해진 목록(enum)에서만 고르게 하고, 해당 없음은 null 대신 "없음"이라는 글자로 받는다.
- **정제:** 리라이팅 문장이 비었거나 120자를 넘으면 원문을 쓰고, 성분 이름은 한글·영문·숫자 20자 이내 규칙에 맞는 것만 남긴다. 하위 질문이 하나뿐이면 `rewritten`과 겹치므로 버린다.
- **임신 여부는 코드로도 확인:** 원문에 임신·임산부·수유·모유가 있으면 모델 답과 상관없이 임신으로 처리한다.
- **폴백:** 호출 실패, 시간 초과(6초), 도구 응답 없음 중 무엇이든 원문 그대로 진행한다. 이때도 임신 확인은 동작하고, 오류 원인은 `fallback_reason`에 남는다.
- **비용 관리:** 같은 문장은 캐시(256개)에서 꺼내 쓰고, 이어지는 질문("그럼 크림으로 추천해줘")은 최근 문장 3개를 함께 보내 풀어 준다.

2.4 점수를 합치는 방법

리라이팅 뒤에도 고민 분류는 KURE-v1과 BGE-M3가 맡는다. 달라진 점은 원문, 리라이팅 문장, 하위 질문을 모두 검색해 점수를 합치는 것이다.

```
s(c) = max( (1 − w)·s_orig(c) + w·s_rw(c),  λ·max_i s_sub,i(c) ),   w = 0.6, λ = 0.9
```

여기서 c는 고민, s는 각 문장으로 검색했을 때의 점수다. 아래 첨자 orig, rw, sub는 각각 원문, 리라이팅 문장, 하위 질문이다. 합친 뒤에는 1등 점수를 1.0으로 맞춘다. 규칙은 두 가지 판단에서 나온다.

- **같은 뜻의 두 문장은 가중 평균한다.** 원문과 리라이팅 문장은 같은 말이므로 평균을 내면, 리라이팅이 틀려도 원문 점수가 받쳐 준다.
- **서로 다른 고민은 최댓값을 쓴다.** 하위 질문은 각자 다른 고민을 찾는 문장이라, 평균을 내면 점수가 묻힌다. 그래서 고민별 최댓값에 0.9를 곱해 반영한다.

가중치 0.6과 0.9는 시작값이며, 평가 스크립트(`eval_rewrite.py --sweep`)로 조정하도록 만들어 두었다.

3. 올리브영 크롤링 AI 에이전트

에이전트는 질문을 보고 실시간 정보가 필요한지 스스로 판단하고, 필요할 때만 올리브영을 수집해 답하는 장치다. 필요 없는 질문은 이전에 쓰던 저장 데이터(DB) 경로로 그대로 흘러간다.

3.1 왜 필요한가

RAG의 장점 중 하나는 외부 지식을 바꿔 끼우면 세상의 변화를 따라갈 수 있다는 점이다(Lewis et al., 2020). 그러나 저장 DB는 수집 시점에 멈춰 있어, 순위·가격·할인·리뷰 수처럼 매일 바뀔 수 있는 값은 답이 낡기 쉽다. "요즘 선크림 순위 알려줘"는 DB에 있는 내용이 아니라 지금의 내용을 묻는 질문이다. 그래서 모든 질문을 크롤러로 보내지 않고, 질문마다 수집이 필요한지를 먼저 판단한다. 모든 질문을 수집으로 보내면 느리고, 비용이 드는 데다, 대상 사이트에 요청이 몰릴 수 있다.

3.2 에이전트의 두 가지 설계 방식

에이전트를 만드는 방식은 크게 두 갈래다. ReAct는 모델이 생각(추론)과 행동(도구 호출)을 번갈아 가며 스스로 다음 단계를 정하는 방식이다(Yao et al., 2022). 반대로 단계와 분기를 미리 그래프로 정해 두고, 분기 지점에서만 모델이 판단하게 하는 방식이 있다. 이 시스템은 뒤에 가까운 방식을 택했다.

비교	모델이 다음 단계를 정함 (ReAct형)	그래프로 단계를 고정 (이번 방식)
유연성	높다. 예상 못 한 질문도 스스로 풀어 간다	낮다. 그래프가 허용한 경로만 간다
예측 가능성	낮다. 호출 횟수·순서가 매번 달라질 수 있다	높다. 최대 단계가 정해져 있다
비용·지연	판단 호출이 반복될 수 있다	판단은 한 번, 수집은 필요할 때만

비교	모델이 다음 단계를 정함 (ReAct형)	그래프로 단계를 고정 (이번 방식)
테스트	경로가 많아 확인하기 어렵다	노드·분기별로 나눠 확인할 수 있다

질문 유형이 "실시간이 필요한가, 아니면 저장 데이터로 충분한가" 두 갈래로 좁아서 유연성보다 예측 가능성을 우선했다. 이 판단은 이 서비스의 질문 분포를 전제로 한 설계 선택이며, 질문이 다양해지면 반복 호출형으로 넓히는 것을 검토해야 한다.

3.3 LangGraph 상태 그래프

LangGraph는 에이전트를 상태 그래프(StateGraph)로 짤 수 있게 해 주는 라이브러리다. 노드는 상태를 갱신하는 함수이고, 엣지는 현재 상태를 보고 다음에 실행할 노드를 정한다. 조건부 엣지를 쓰면 실행 중의 상태에 따라 경로가 갈린다(LangGraph 문서). 이 시스템의 그래프는 세 노드와 하나의 분기로 이루어졌다.

1. **analyze (질문 분석):** Claude가 질문을 보고 실시간 정보가 필요한지, 필요하다면 의도(랭킹·신제품·가격)와 카테고리(선크림 등)를 정한다.
2. **조건부 분기:** 실시간이 필요하면 crawl로, 아니면 그래프를 바로 끝내고 기존 추천 경로(리라이팅 → 고민 검색 → 제품 거르기)로 넘긴다.
3. **crawl (수집):** 올리브영에서 가격·평점·리뷰·성분을 가져온다.
4. **summarize (정리):** 수집한 제품을 카드 형식으로 정리해 답한다.

각 노드가 실행시간과 판단 내용을 기록(trace)하기 때문에, 화면의 "추천 근거 보기"에서 어느 단계를 거쳤는지 확인할 수 있다.

3.4 수집 결과를 다루는 방식과 위험

- **외부 데이터는 명령이 아니라 자료로 취급한다.** 수집한 리뷰·설명에 "이전 지시를 무시하라"는 문장이 섞여 있으면 모델이 이를 따를 수 있다. 이를 간접 프롬프트 인젝션이라 부른다(Greshake et al., 2023). 그래서 가격·평점·성분처럼 정해진 필드만 뽑아 코드가 직접 화면에 넣고, 자유 문장인 리뷰 원문은 모델에 그대로 넣지 않는다.
- **수집 실패는 기존 경로로 되돌린다.** 사이트 차단·시간 초과·구조 변경으로 수집이 비면 저장 DB로 답할 수 있게 두는 것이 안전하다. (적용 여부는 실제 에이전트 코드에서 확인이 필요하다.)
- **대상 사이트 규칙을 지킨다.** 크롤링 전에 이용약관과 robots.txt를 확인하고, 요청 빈도를 낮춰 추가 부하를 주지 않아야 한다. 이 보고서는 이 항목의 실제 확인 결과를 담지 않는다.

4. RAG 이전 가드레일

가드레일은 고객 입력이 검색·모델 호출·크롤러에 닿기 전에 위험한 입력을 막거나 고치는 검사 단계다. 이 시스템은 입력 직후, 파이프라인의 가장 앞에 둔다.

4.1 왜 RAG 이전에 두는가

- **비용과 지연:** 차단할 입력은 임베딩 검색과 LLM 호출을 하지 않고 바로 답할 수 있다.
- **피해 범위 축소:** 인젝션 시도가 수집 도구나 추천 로직에 닿기 전에 끝난다.
- **개인정보 유출 방지:** 외부 API와 로그로 나가기 전에 번호·이메일을 가린다. 로그에는 고객 문장을 남기지 않는다(리라이팅 폴백 로그 참고).

4.2 네 가지 위험과 처리 방식

검사는 위험한 것부터 순서대로 하고, 먼저 걸린 것을 따른다.

순서	검사	예시	처리	고객에게
0	빈 입력, 500자 초과	(빈 칸), 너무 긴 글	차단	입력하거나 줄여 달라고 안내
1	응급·위험 증상	"숨쉬기가 힘들어요"	차단	119·응급실, 피부과 상담 안내
2	프롬프트 인젝션	"이전 지시를 모두 무시해"	차단	처리하지 않았다고 안내
3	피부와 무관한 입력	"주식 추천해줘"	차단	피부 고민을 알려 달라고 안내
4	개인정보	전화번호·주민번호·이메일·카드번호	통과(마스킹)	가렸다고 알림

위험 증상. 화장품 추천 챗봇은 의료 상담이 아니다. 호흡 곤란, 심한 통증, 의식 이상, 심한 출혈·화상, 시야 이상, 고열과 전신 발진 같은 표현이 있으면 추천을 멈추고 의료진 확인을 권한다. 증상을 진단하거나 제품을 대안으로 제시하지 않는 것이 원칙이다.

프롬프트 인젝션. OWASP는 공격자가 정교하게 만든 입력으로 모델을 조종해 의도하지 않은 동작을 하게 만드는 것을 프롬프트 인젝션이라 정의한다(OWASP LLM01). 모델은 지시와 데이터를 구조적으로 구분하지 못해서, 입력에 섞인 명령이 시스템 지침처럼 작동할 수 있다. 사용자가 직접 쓰는 직접 인젝션과, 검색이나 수집으로 들어오는 외부 문서에 숨긴 간접 인젝션이 있다(Greshake et al., 2023). 가드레일은 앞의 직접 인젝션을 막고, 뒤의 간접 인젝션은 3장의 수집 결과 취급 규칙이 맡는다.

주제 이탈. 피부와 무관한 질문에 답하면 비용이 늘고 서비스의 신뢰도가 흐려진다. 피부 관련 단어가 없거나 제외 단어(주식, 날씨 등)가 있으면 돌려보낸다. 인사말은 통과시킨다.

개인정보. 고객이 피부 고민에 번호를 섞어 쓰는 실수는 흔하다. 그래서 문장 전체를 막지 않고 번호만 "[전화번호]"처럼 바꾸어 나머지를 처리한다. 필요 최소한만 다룬다는 원칙에 맞춘 설계다.

4.3 규칙 기반 검사의 장단점

이번 가드레일은 정규식·키워드 규칙으로 만들었다. LLM 분류기와 비교한 이유와 한계는 다음과 같다.

비교	규칙 기반 (이번)	LLM 분류기
속도·비용	호출 없이 바로 판정	호출마다 지연과 비용
설명 가능성	어느 규칙에 걸렸는지 바로 확인	판단 이유를 추가로 해석해야 함
새로운 표현	바꿔 쓴 말·오타·다른 언어는 놓칠 수 있다	대체로 유연하다
오탐	키워드가 공교롭게 걸릴 수 있다	문맥을 보고 줄일 수 있다
자체 위험	없음	분류기도 인젝션의 대상이 될 수 있다

응급 증상과 개인정보처럼 빠짐없이 걸러야 하는 항목은 설명 가능한 규칙이 유리하다. 다만 규칙은 완전하지 않아, 방어를 여러 겹으로 두는 것이 중요하다. 뒤따르는 단계가 그 역할을 한다.

- 리라이팅 스키마가 출력을 정해진 필드로 제한하고, 프롬프트 태그를 깨는 `<` `>` 글자를 제거한다.
- 제품 추천은 저장된 데이터의 필드만 쓰므로, 모델이 흔들리더라도 할 수 있는 일이 작다.
- 화면은 텍스트로만 출력해 답변에 섞인 코드가 실행되지 않는다.

한계도 있다. 키워드 목록은 적은 표본에서 만든 것이라 새로운 표현이 빠질 수 있고, "눈이 아파요"처럼 위험 표현과 일상 표현이 가까운 문장은 오탐 가능성이 있다. 실제 고객 문장으로 오탐·미탐을 측정한 결과는 아직 없다.

5. 성분 데이터 보완과 제품 추천

제품 추천의 정확도는 제품의 전성분 데이터가 얼마나 빠짐없는지에 달려 있다. 이 시스템은 고민에 맞는 성분을 제품 전성분에서 찾아 추천하고, 피해야 할 성분이 들어 있는지도 같은 데이터로 판단하기 때문이다. 지난주까지 부족했던 성분표를 이번 주에 보완했고, 보완한 데이터는 아래 두 역할에 쓰인다.

5.1 고민에서 제품까지

1. 고민별로 도움이 되는 성분 사전을 둔다. 예: 홍조는 병풀추출물·판테놀, 번들거림은 나이아신아마이드·징크 PCA.
2. 제품의 전성분과 사전을 대조해 제품마다 "도움 성분"을 찾는다.
3. 피부 타입, 제품 종류, 제외 성분 조건으로 제품을 거른다.
4. 남은 제품을 점수로 정렬해 상위 5개를 보여 준다.

시연 구현의 점수는 다음과 같다(실제 시스템의 가중치는 다를 수 있다).

```
score(p) = Σ_{c∈M(p)} w_c  +  0.05·n_ing(p)  +  0.3·(r_p − 4.0)
```

여기서 M(p)는 제품 p가 도움 성분을 가진 고민의 집합, w_c는 2장에서 구한 고민 점수, n_ing는 찾은 도움 성분의 수, r_p는 평점이다. 고민 점수가 가중치로 들어가기 때문에 앞장의 리라이팅이 추천 순서까지 영향을 준다. 동점이면 평점, 그다음 제품 번호 순이다.

5.2 성분 이름을 맞추는 법

제외 성분을 "글자 일부가 포함되면 거른다"는 규칙으로 판단하면 오탐이 생긴다. 예를 들어 에탄올을 피하려고 했는데 페녹시에탄올(방부제)까지 걸러지는 식이다. 그래서 이름이 같거나 앞부분이 맞는 경우만 같은 성분으로 본다. 파라벤처럼 끝이 같은 성분군(메틸파라벤 등)만 끝으로도 맞춘다. 고객이 쓰는 말은 표준 성분명으로 바꾼다(예: 알코올→에탄올, 인공향→향료, BHA→살리실).

5.3 제외 성분과 안전 규칙

제외 성분은 고객이 직접 말한 것과, 시스템이 안전 규칙으로 더하는 것을 합친다. 안전 규칙은 안전 쪽으로 치우친 시작값이고, 실제 운영 전에 전문가(피부과·약사) 검토를 받아야 한다.

조건	더해지는 제외 성분	비고
고객이 직접 말한 성분	말한 성분 그대로(표준명 변환)	최대 8개
민감성 또는 민감 성향 고민(민감·자극, 홍조, 열감, 장벽 손상)	에탄올, 향료	자극 가능성을 줄이는 보수적 규칙
임신·수유	레티놀, 레티닐, 살리실	보수적 규칙, 전문가 검토 필요

두 가지가 충돌할 수 있다. 주름 고민에는 레티놀이 도움 성분이지만, 임신 중이면 제외 성분이다. 이 경우 **제외가 도움보다 우선**한다. 시연에서 "건성인데 눈 밑이 어둡고 팔자주름도 깊어졌어요. 알코올 없는 크림으로 추천해줘. 임신 중이에요"를 입력하면 후보 4개 → 제품 종류(크림) 3개 → 피부 타입(건성) 3개 → 성분 제외 후 2개로 줄어들고, 레티놀이 든 크림 하나가 빠진다.

5.4 성분 데이터 보완이 중요한 이유

전성분이 비어 있거나 덜 채워진 제품은 두 방향으로 추천을 흔든다.

- **도움 성분을 놓친다:** 좋은 제품이 점수가 낮아져 후보에서 밀린다.
- **제외 성분을 못 거른다:** 성분표에 없다는 이유로 피해야 할 성분이 든 제품이 통과한다. 두 번째가 더 위험하다. 임신 중인 고객에게 레티놀이 든 제품이 나갈 수 있기 때문이다.

그래서 제외 조건이 있을 때는 전성분이 없는 제품을 추천에서 빼거나 "성분 정보 없음"으로 표시하는 정책을 정해야 한다(이번 시연에서는 모든 제품에 전성분을 두었으므로 적용하지 않았다). 화면은 제품 카드에 "전성분 N종 확인"과 "제외 성분은 전성분에 없어요"를 보여 주어, 성분 데이터가 추천에 쓰였다는 사실을 고객이 확인하게 했다.

열린 질문: 성분표를 확보한 제품의 비율이 보완 전후로 몇 %인지 수치를 확인해 이 절에 넣을 것. (이 보고서 작성 시점에는 측정값이 없다.)

6. 구현 및 검증

이 작업은 세 기능의 흐름과 화면을 끝까지 실행해 본 것이고, 실제 모델을 써서 정확도를 측정한 것은 아니다. 시연 환경에는 Claude API 키, KURE-v1·BGE-M3 모델 내려받기, 올리브영 접속이 모두 없어서 해당 부분은 대체 구현으로 동작시켰다.

6.1 실제 구현과 대체 구현

구성 요소	실제 설계	시연에서 쓴 것
쿼리 리라이팅 코드	프롬프트·도구 스키마·검증·폴백·캐시, 점수 합치기	업로드하신 코드 그대로 실행
Claude 호출	Claude(기본 claude-haiku-5-5)	규칙 기반 시뮬레이터가 같은 인터페이스로 대신
고민 검색	KURE-v1 + BGE-M3 임베딩	글자 2-gram TF-IDF 대체 검색기
가드레일	입력 검사·마스킹	규칙 기반으로 새로 작성 (업로드한 압축 파일에 없었음)
올리브영 에이전트	Claude 판단 + 실제 크롤링	LangGraph 그래프는 실제 실행, 판단은 규칙, 수집은 가상 샘플
제품 데이터	실제 제품·전성분 DB	가상 제품 32개
화면	기존 챗봇 프론트엔드	기존 화면에 새 화면을 덧붙임

화면에는 "시뮬레이터 · 샘플 데이터" 표시와 안내문을 넣어, 대체 구현이 실제 성능으로 오해되지 않게 했다. 가상 제품은 실제 올리브영 링크를 담지 않는다.

6.2 확인한 것

- **단위 테스트:** `test_query_rewriter.py`의 36개 중 35개가 통과했고, 1개는 필요한 CSV가 없어 건너뛰었다. 가드레일을 추가한 뒤에도 이 테스트가 그대로 통과한다.
- **종단 흐름:** 화면의 예시 문장 13개를 모두 보내 오류 없이 응답이 오는 것을 확인했다. 이 중 처리 흐름은 네 갈래다: 리라이팅 후 검색, 올리브영 수집, 가드레일 차단, 개인정보 마스킹 후 통과.
- **동작 원리:** 아래 예는 리라이팅이 점수를 어떻게 바꾸는지 보여 주는 시연 결과다. 대체 검색기와 시뮬레이터로 구한 값이므로 실제 모델의 성능을 대표하지 않는다.

입력	원문만 검색	리라이팅 반영
볼때기가 맨날 뻘개요 ㅠ	고민을 찾지 못함	홍조 1.00, 장벽 손상 0.60
건성인데 눈 밑이 어둡고 팔자주름도 깊어졌어요 (하위 질문 2개)	주름 1.00, 다크서클 0.65	주름 1.00, 다크서클 0.90
건성 피부에 맞는 수분크림 추천해줘	복합성 유수분 불균형 1.00, 속건조 0.66	속건조 1.00, 복합성 유수분 불균형 0.46

6.3 확인하지 못한 것

- 실제 Claude와 KURE-v1·BGE-M3로 고민 분류 정확도(top-1, top-3)가 얼마나 올랐는지.
- 리라이팅 호출의 지연 시간과 비용, 폴백이 나온 비율.
- 실제 올리브영 크롤링의 안정성과 이용약관 확인 결과.
- 가드레일의 오탐·미탐률, 안전 규칙 성분 목록의 전문가 검토.
- 성분표 확보 비율의 보완 전후 수치.

7. 결론 및 향후 과제

세 개선은 서로 다른 지점의 한계를 맡고 파이프라인의 서로 다른 위치에 놓여, 입력 검사 → 최신 정보 판단 → 문장 이해 → 성분 기반 추천으로 이어진다. 세 개선에 공통으로 쓴 설계 원칙은 다섯 가지다.

- **LLM의 출력을 그대로 믿지 않는다.** 스키마로 모양을 정하고, 코드가 다시 검사하고, 실패하면 원문으로 되돌아간다.
- **비싼 경로는 필요할 때만 탄다.** 가드레일이 먼저 막고, 에이전트가 수집 필요성을 판단하고, 캐시가 같은 문장의 재호출을 줄인다.
- **안전이 관련성보다 우선한다.** 응급 증상은 추천보다 앞서고, 제외 성분은 도움 성분보다 앞서며, 개인정보는 외부로 나가기 전에 가린다.
- **외부 데이터는 명령이 아니라 자료다.** 수집 결과는 정해진 필드만 뽑아 쓴다.
- **한 계층에 기대지 않는다.** 규칙 가드레일이 놓쳐도 스키마 제한과 제품 데이터 기반 추천이 피해를 줄인다.

향후 과제는 "대체 구현으로 본 동작을 실제 구성에서 측정하는 것"이 중심이다.

- [ ] 실제 Claude와 KURE-v1·BGE-M3로 고민 분류 정확도(top-1, top-3)를 측정하고, `eval_rewrite.py --sweep`으로 가중치 0.6·0.9를 조정한다.
- [ ] 리라이팅 호출의 지연 시간, 비용, 폴백 비율을 측정한다.
- [ ] 가드레일의 오탐·미탐률을 실제 고객 문장으로 재고, 필요하면 분류기를 덧붙인다.
- [ ] 안전 규칙 성분 목록(에탄올·향료, 레티놀·레티닐·살리실)을 전문가에게 검토받는다.
- [ ] 올리브영 이용약관·robots.txt를 확인하고, 수집 실패 시 저장 DB로 되돌리는지 확인한다.

- [ ] 성분표 보유 비율의 보완 전후 수치를 확인해 5장에 넣고, 전성분이 없는 제품의 처리 정책을 정한다.

참고문헌

본문의 이론 설명에 쓴 자료다. 접속일은 2026-11-11이다.

1. Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401.
2. Ma, X., Gong, Y., He, P., Zhao, H., Duan, N. (2023). Query Rewriting for Retrieval-Augmented Large Language Models. EMNLP 2023. arXiv:2305.14283.
3. Chen, J. et al. (2024). BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. arXiv:2402.03216.
4. NLP&AI Lab, Korea University. KURE-v1 model card. Hugging Face.
5. Yao, S. et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629.
6. Greshake, K. et al. (2023). Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. arXiv:2302.12173.
7. OWASP GenAI Security Project. LLM01: Prompt Injection.
8. Anthropic. Define tools (tool_choice). Claude API documentation.
9. LangChain. LangGraph: Build sequences, branches, and loops. Documentation.